# 数据与模型接口交接说明

| 项目 | 当前约定 |
| --- | --- |
| 当前时间粒度 | 日级 |
| 数据处理 | 独立 `data-worker` 进程异步读取、校验、清洗并发布 |
| 模型调用 | 独立 `model-api` 进程使用 FastAPI 提供 REST API |
| 模型运行方式 | 只加载已训练模型推理，生产 API 不训练模型 |
| 数据交换 | XML/DT 使用文件目录；原 CSV 部分改为从 DB 读取 |
| 调用方 | 南瑞方业务系统通过 REST API 调用 |

## 引言

本交付由两个相互独立的进程组成：

1. `data-worker` 由华工方负责，异步读取 XML/DT 与 DB 业务表，执行校验、关联、日级聚合、历史特征加工和版本发布。
2. `model-api` 由华工方负责，只读取 Worker 已发布的数据，调用已训练的设备模型、站点模型和经济调度模型，通过 FastAPI 向南瑞方返回 JSON。

```
XML/DT 文件 + DB 业务表
              |
              v
  data-worker（异步清洗）
              |
              v
   版本化设备/站点日数据
              |
              v
 model-api（FastAPI + 已模型）
              |
              v
       南瑞方业务系统
```

API 请求不会在线扫描原始大表，也不会在线重新训练模型。数据处理失败时不更新当前活跃版本，API 仍读取上一个完整版本。

# 1. 数据部分

## 1.1 输入数据清单

| 序号 | 数据名称 | 原格式 | 接入方式 | 当前时间粒度 | 用途 |
|---|---|---|---|---|---|
| 1 | CIM 模型数据 | XML/RDF | 文件 | 模型快照，按模型更新 | 设备、站点、测点及关系 |
| 2 | 设备运行状态数据 | DT | 文件 | 原文件内含时间；当前按日聚合 | 电流、电压、功率和开关状态 |
| 3 | 主网设备数据 | CSV | DB | 日级/台账变化时 | 设备基础信息和设备匹配 |
| 4 | 缺陷数据 | CSV | DB | 日级累计历史 | 历史缺陷特征 |
| 5 | 检修单主表数据 | CSV | DB | 日级累计历史 | 检修任务信息 |
| 6 | 检修单设备明细数据 | CSV | DB | 日级累计历史 | 检修单与设备关联 |
| 7 | 变电跳闸记录 | CSV | DB | 日级累计历史 | 变电设备跳闸历史 |
| 8 | 输电跳闸记录 | CSV | DB | 日级累计历史 | 输电线路跳闸历史 |

建议 XML 文件名为 `zs_dicp_model-YYYYMMDD.xml`；DT 文件保持现有 `*.DT` 命名。原 CSV 对应的 DB 逻辑表为：

```
DM_主网设备
DM_缺陷数据
DM_检修单主表_JXSQD
DM_检修单从表_TD_JXSBLB
DM_变电跳闸记录
DM_输电跳闸记录
```

DB 中可以是表或视图，实际 schema 和名称通过 `DB_TABLE_MAP` 映射，不要求数据中心使用上述物理表名。XML 和 DT 仍由文件目录接入，不从 DB 读取。

## 1.2 XML/DT 放置路径

每个批次使用独立的 `batch_id`，放入共享运行目录：

```
api_service/runtime/inbox/<batch_id>/
├── xml/
│   └── zs_dicp_model-YYYYMMDD.xml
├── dt/
│   ├── *.DT
│   └── *.DT
└── READY.json
```

当前约束：

| 项目 | 约束 |
| --- | --- |
| `batch_id` | 1 ~ 64 字符，只允许字母、数字、`.`、`_`、`-` |
| XML | 每批次必须且只能有 1 个非空 `.xml` 文件 |
| DT | 每批次至少 1 个非空 `.DT` 文件 |
| 站点名称 | 当前清洗规则保留具有中文站名的正式站点，过滤测试站和空站名 |
| `business_date` | 必须存在于本批次处理后的设备日和站点日数据中 |

## 1.3 如何触发清洗

### 自动触发

先上传 XML/DT，确认文件全部写入完成后，最后放入 `READY.json`。不能在大文件尚未写完时提前生成完成标记。

> *示例说明：下列完成标记会让 Worker 以基础运行数据模式处理 `demo-raw-20260201` 批次。JSON 中不写注释，因此可直接运行。*

```
{
  "request_id": "auto-demo-raw-20260201",
  "business_date": "2026-02-05",
  "run_mode": "operation_only"
}
```

Worker 默认每 2 秒扫描一次 `READY.json`，根据 `request_id` 幂等建立任务。

### API 手动触发

历史补数或人工重跑可调用 `POST /api/data-jobs`。该 API 只创建任务，真正清洗仍由独立 Worker 执行。详细报文见 2.4 节。

## 1.4 处理模式

| run_mode | 输入 | 用途 | 输出质量说明 |
| --- | --- | --- | --- |
| `operation_only` | XML + DT | 文件联调、基础运行评估 | 可运行，但缺陷/跳闸/检修特征不完整，设备结果可返回 `data_quality=warning` |
| `full` | XML + DT + 6 类 DB 业务数据 | 生产日级评估和家族预警 | 从 DB 导出等价数据，补充设备历史、家族三元组和家族统计特征 |

## 1.5 异步处理过程

Worker 实际执行顺序如下：

| 顺序 | 任务阶段 | 处理内容 |
| --- | --- | --- |
| 1 | `validating` | 检查批次目录、XML/DT 数量和空文件 |
| 2 | `extracting_xml` | 从 XML 解析对象和关系 |
| 3 | `merging_dt` | 将 DT 测点值按 XML 关系关联到设备和站点 |
| 4 | `building_device_daily` | 形成设备日运行宽表 |
| 5 | `exporting_database` | `full` 模式下从 DB 读取 6 类业务数据 |
| 6 | `building_device_model_table` | 形成设备模型基础表 |
| 7 | `building_station_daily` | 按站点聚合设备日数据 |
| 8 | `enriching_device_history` | `full` 模式下加工缺陷、跳闸、检修、家族三元组和家族历史 |
| 9 | `publishing` | 校验业务日期，发布新的设备/站点数据版本 |

数据处理与 API 查询不在同一进程。Worker 使用 SQLite 任务表原子领取待处理任务，避免同一任务被同时处理。

## 1.6 处理结果存储与模型读取

```
api_service/runtime/
├── state/service.db                # 异步任务状态
├── jobs/<job_id>/                  # 当次中文件、日志和果
└── published/
    ├── versions/<feature_version>/ # 不可变设备/站点特征文件
    └── active.json                 # 当前活跃版本指针
```

Worker 先写入新版本目录，再原子替换 `active.json`。`model-api` 每次读取当前活跃版本，然后使用模型产物内保存的字段顺序和预处理器进行推理。容器部署时 API 和 Worker 共享该持久化目录。

## 1.7 DB 对接

只有原 CSV 部分从 DB 读取。当前代码已使用 `DATABASE_URL` 和 `DB_TABLE_MAP` 保留数据库接入点，Worker 将数据库表/视图转换为现有清洗脚本可读的等价文件，再执行后续流程。

```
DATABASE_URL=postgresql+psycopg://user:password@db-host:5432/database
DB_TABLE_MAP={"dm_device":"schema.dm_device","defect":"schema.dm_defect","maintenance_main"
:"schema.dm_maintenance_main","maintenance_device":"schema.dm_maintenance_device","trip_sub
station":"schema.dm_trip_substation","trip_line":"schema.dm_trip_line"}
```

实际联调前需由华工方与南瑞方确认：

- DB 类型与版本，例如 Oracle、PostgreSQL、MySQL 或 SQL Server。
- 表或视图名、schema、账号权限和对应的 Python 驱动。
- 字符集、时区、空值、历史数据范围和增量字段。
- 读取账号使用只读权限，密码由部署平台注入，不进入代码和 Git。

家族缺陷预警使用主网设备台账生成“厂家 + 型号 + 批次”三元组。批次按以下顺序取得：优先读取 DB 中的`生产批次`、`设备批次`、`批次`或`出厂批次`字段；没有独立批次字段时，将`出厂日期`归一为 `YYYYMM`，并在接口的 `batch_source` 字段中标记为推导值。厂家、型号或批次任一缺失时，该设备不进入家族预警统计。

## 1.8 秒级/分钟级数据说明

当前设备和站点模型均为日级模型。DT 文件包含 `Time`，但当前流程会将同一设备同一天的多条记录合并为一条设备日记录，保留输入顺序中最后遇到的有效值。

当前设备模型和站点模型均按日进行评估。各类输入数据在现有模型中的时间粒度如下：

| 输入数据 | 当前时间粒度 | 秒/分钟级接入说明 |
| --- | --- | --- |
| `zs_dicp_model-YYYYMMDD.xml` | 日级模型快照，模型变化时更新 | 不需要按秒或分钟重复提供；拓扑变化时及时更新 |
| `*.DT` | 当前模型按日聚合使用 | 需要提供秒级或分钟级连续测点数据，是高频评估的核心输入 |

| 输入数据 | 当前时间粒度 | 秒/分钟级接入说明 |
| --- | --- | --- |
| DM_主网设备.csv | 设备台账，无连续时间粒度 | 不需要按秒或分钟生成；设备属性变化时更新 |
| DM_缺陷数据.csv | 事件级记录，当前模型按日统计 | 不需要每秒重复；新增或变更缺陷时及时增量提供 |
| DM_检修单主表_JXSQD.csv | 检修单级记录，当前模型按日统计 | 不需要每秒重复；检修状态变化时及时提供 |
| DM_检修单从表_TD_JXSBLB.csv | 检修设备关联记录，当前模型按日统计 | 不需要每秒重复；设备关联变化时提供 |
| DM_变电跳闸记录.csv | 事件级记录，当前模型按日统计 | 不需要每秒快照；事件发生时间应尽量精确到秒 |
| DM_输电跳闸记录.csv | 事件级记录，当前模型按日统计 | 不需要每秒快照；事件发生时间应尽量精确到秒 |

说明：表中 DM 文件名用于标识既有 CSV 数据口径，不代表生产环境继续通过 CSV 文件接入；正式部署时按第 1.7 节从 DB 表或视图读取。

如后续建立秒级或分钟级模型，需提高频率的是第 2 类 DT 运行数据，至少保留：

```
Time
measurement_id (DT 中的 ID)
Value 或 Status
量量 (如网可提供)
```

XML 拓扑和 DM 业务数据不需要强制提高到秒级，它们按各自更新频率维护。秒级/分钟级能力需新增保留原始时间戳的处理分支并重新训练对应模型，不能将当前日级模型直接宣称为秒级预测。

# 2. API 接口部分

## 2.1 通用约定

| 项目 | 约定 |
| --- | --- |
| 协议 | REST API，当前服务使用 HTTP；生产 HTTPS 由网关终止 |
| 默认端口 | `8000` |
| 格式 | `application/json; charset=utf-8` |
| 日期 | `YYYY-MM-DD` |
| 请求追踪 | POST 使用请求体 `request_id`；GET 可使用 `X-Request-ID` 请求头 |
| 训练 | 所有 API 均不触发模型训练 |
| 单次 ID 数量 | POST 中显式 ID 列表最多 100 个；不传则返回该日全部对象，默认同步上限 10000 条 |

所有成功响应使用统一外层：

| 字段 | 类型 | 含义 |
| --- | --- | --- |
| `code` | string | `0` 表示成功 |
| `message` | string | 处理结果说明 |
| `request_id` | string | 请求追踪标识 |
| `data` | object | 接口业务数据 |

状态等级统一为：

| state_level | state_name | 含义 |
| --- | --- | --- |
| 0 | `normal` | 正常 |
| 1 | `watch` | 关注 |
| 2 | `abnormal` | 异常 |
| 3 | `high_risk` | 高风险 |

> *报文说明：本章示例使用随代码交付的演示数据。启动 API 和 Worker 并执行 `python3 -m api_service.demo.prepare_demo_data` 后，下列 `curl` 可在项目根目录直接联调。JSON 内不加非标准注释；每个示例前的“示例说明”用于说明用途。`request_id`、`job_id`、时间、数据版本和模型分数以实际返回为准。*

## 2.2 存活检查

```
GET /health/live
```

- 请求体：无。
- 接口约束：可选请求头 `X-Request-ID`。
- 响应体：HTTP 200；`data.status=up` 表示 API 进程能响应 HTTP，不代表数据已发布。

> *示例说明：检查 API 进程是否存活。*

```
curl -H "X-Request-ID: demo-live-001" http://127.0.0.1:8000/health/live
{
  "code": "0",
  "message": "success",
  "request_id": "demo-live-001",
  "data": {"status": "up", "service": "model-api"}
}
```

## 2.3 就绪检查

```
GET /health/ready
```

- 请求体：无。
- 接口约束：必须已有一个成功发布的数据版本。
- 响应体：就绪时返回 HTTP 200 和当前特征版本；未就绪时返回 HTTP 503，`code=FEATURE_DATA_NOT_READY`。

| 响应字段 | 含义 |
| --- | --- |
| `status` | `ready` 表示可接收模型请求 |
| `feature_version` | 当前活跃特征版本 |
| `business_date` | 该版本标识的业务日期 |

> *示例说明：业务系统发流量前检查模型数据是否就绪。*

```
curl http://127.0.0.1:8000/health/ready
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "req-6b8302c231704896",
  "data": {
    "status": "ready",
    "feature_version": "20260207-demo-seed-081008-f053a8",
    "business_date": "2026-02-07"
  }
}
```

## 2.4 创建数据清洗任务

```
POST /api/data-jobs
```

### 请求体

| 字段 | 类型 | 必填 | 约束与含义 |
| --- | --- | --- | --- |
| `request_id` | string | 是 | 1 ~ 96 字符，幂等键 |
| `batch_id` | string | 是 | 必须已存在于 `runtime/inbox/` |
| `business_date` | date | 是 | 本批次业务日期 |
| `run_mode` | string | 是 | `operation_only` 或 `full` |
| `force` | boolean | 否 | 默认 `false`；`true` 创建一次新的重试任务 |

### 响应体

| **`data`** 字段 | 含义 |
| --- | --- |
| `job_id` | Worker 任务唯一标识 |
| `batch_id` / `business_date` / `run_mode` | 本次任务的输入标识 |
| `status` | `pending`、`running`、`succeeded` 或 `failed` |
| `current_stage` | 当前处理阶段 |
| `created_at` / `started_at` / `finished_at` | 任务时间 |
| `result` | 成功后的数据版本和行数摘要 |
| `error_message` | 失败时的错误信息 |

### 接口约束

- `batch_id` 目录必须已存在，且 XML/DT 文件已完整写入。

- 成功接收：HTTP 202。
- 相同 `request_id` 且 `force=false` 时返回原任务，不重复创建。
- API 返回 202 不代表清洗已完成，必须继续查询任务状态。

> *示例说明：手动触发已放入目录的 XML/DT 演示批次。*

```
curl -X POST http://127.0.0.1:8000/api/data-jobs \
  -H "Content-Type: application/json" \
  -d '{
    "request_id": "demo-cleaning-001",
    "batch_id": "demo-raw-20260201",
    "business_date": "2026-02-05",
    "run_mode": "operation_only",
    "force": false
  }'
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "demo-cleaning-001",
  "data": {
    "job_id": "job-20260811090000-a1b2c3d4",
    "batch_id": "demo-raw-20260201",
    "business_date": "2026-02-05",
    "run_mode": "operation_only",
    "status": "pending",
    "current_stage": "queued",
    "created_at": "2026-08-11T09:00:00+00:00",
    "started_at": null,
    "finished_at": null,
    "result": null,
    "error_message": null
  }
}
```

## 2.5 查询数据清洗任务

```
GET /api/data-jobs/{job_id}
```

- 请求体：无。
- 接口约束：路径参数 `job_id` 必须使用 2.4 节实际返回的任务 ID；任务不存在时返回 HTTP 404。
- 响应体：HTTP 200，字段与 2.4 节一致。

> *示例说明：查询已完成清洗任务的发布结果。*

```
JOB_ID=$(curl -sS -X POST http://127.0.0.1:8000/api/data-jobs \
  -H "Content-Type: application/json" \
  -d '{"request_id":"demo-cleaning-001","batch_id":"demo-
raw-20260201","business_date":"2026-02-05","run_mode":"operation_only","force":false}' \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["data"]["job_id"])')
curl "http://127.0.0.1:8000/api/data-jobs/${JOB_ID}"
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "req-b835b16e8ec54bad",
  "data": {
    "job_id": "job-20260811090000-a1b2c3d4",
    "batch_id": "demo-raw-20260201",
    "business_date": "2026-02-05",
    "run_mode": "operation_only",
    "status": "succeeded",
    "current_stage": "published",
    "created_at": "2026-08-11T09:00:00+00:00",
    "started_at": "2026-08-11T09:00:01+00:00",
    "finished_at": "2026-08-11T09:00:02+00:00",
    "result": {
      "feature_version": "20260205-demo-raw-20260201-090002-a1b2c3",
      "device_rows": 10,
      "station_rows": 5,
      "business_date": "2026-02-05",
      "run_mode": "operation_only",
      "log_file": "jobs/job-20260811090000-a1b2c3d4/pipeline.log"
    },
    "error_message": null
  }
}
```


## 2.6 设备日分级评估

```
POST /api/device/assessment
```

### 请求体

| 字段 | 类型 | 必填 | 约束与含义 |
|---|---|---|---|
| `request_id` | string | 是 | 1 ~ 64 字符 |
| `assessment_date` | date | 是 | 已发布数据中的日期 |
| `device_ids` | string[] / null | 否 | 1 ~ 100 个不重复设备 ID；不传或 `null` 表示该日全部设备 |

### 响应体

| 字段 | 含义 |
|---|---|
| `assessment_date` | 评估日期 |
| `model_name` / `model_version` | 设备模型名称和模型文件校验版本 |
| `items[].device_id` / `station_id` | 设备和所属站点 |
| `items[].state_level` / `state_name` | 四级分级评估结果 |
| `items[].confidence` | 预测类别的最高概率 |
| `items[].high_risk_probability` | `abnormal` 和 `high_risk` 两类概率之和 |

| 字段 | 含义 |
| --- | --- |
| `items[].dominant_discriminator` | 当前设备权重最高的判别器 |
| `items[].discriminator_weights` | 运行、历史、检修和设备画像四个判别器权重 |
| `items[].data_quality` | `pass` 或 `warning`，表示当前输入完整度 |

### 接口约束

- `assessment_date` 必须存在于已发布数据中；`device_ids` 最多 100 个、不得重复，未传时返回该日全部设备。
- 日期或设备不存在时返回 HTTP 404；请求体校验失败时返回 HTTP 422。

> *示例说明：对两台设备执行 2026-02-07 单日分级评估。*

```
curl -X POST http://127.0.0.1:8000/api/device/assessment \
  -H "Content-Type: application/json" \
  -d '{
    "request_id": "demo-device-001",
    "assessment_date": "2026-02-07",
    "device_ids": ["AFZ.511C", "AFZ.512C"]
  }'
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "demo-device-001",
  "data": {
    "assessment_date": "2026-02-07",
    "model_name": "DAMD-Net",
    "model_version": "9e1e625b5935",
    "items": [
      {
        "device_id": "AFZ.511C",
        "station_id": "SS-AFZ",
        "state_level": 2,
        "state_name": "abnormal",
        "confidence": 0.931702,
        "high_risk_probability": 0.966921,
        "dominant_discriminator": "史事件判别器",
        "discriminator_weights": {
          "operation": 0.058926,
          "history": 0.765558,
          "maintenance": 0.169033,
          "family_profile": 0.006482
        },
        "data_quality": "pass"
      }
    ]
  }
}
```

## 2.7 单设备单日查询

```
GET /api/device/query?assessment_date=YYYY-MM-DD&device_id=...
```

- 请求体：无。

- 接口约束：查询参数 `assessment_date` 和 `device_id` 均必填；设备或日期无数据时返回 HTTP 404。
- 响应体：与 2.6 节相同，`items` 只有该设备。

> *示例说明：查询单台设备的单日结果。*

```
curl "http://127.0.0.1:8000/api/device/query?assessment_date=2026-02-07&device_id=AFZ.511C"
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-9ee7d79bf0d54516",
  "data": {
    "assessment_date": "2026-02-07",
    "model_name": "DAMD-Net",
    "model_version": "9e1e625b5935",
    "items": [{
      "device_id": "AFZ.511C",
      "station_id": "SS-AFZ",
      "state_level": 2,
      "state_name": "abnormal",
      "confidence": 0.931702,
      "high_risk_probability": 0.966921,
      "dominant_discriminator": "历史事件判别器",
      "discriminator_weights": {
        "operation": 0.058926,
        "history": 0.765558,
        "maintenance": 0.169033,
        "family_profile": 0.006482
      },
      "data_quality": "pass"
    }]
  }
}
```


## 2.8 设备时间区间画像

```
GET /api/device/profile?device_id=...&start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
```

- 请求体：无。
- 接口约束：`device_id`、`start_date`、`end_date` 均必填，开始日期不得晚于结束日期，且区间内至少存在一条该设备的已发布日数据。
- 响应体：对区间内已发布的设备日结果进行聚合，不是另一个训练模型；`data` 字段如下。

| 响应字段 | 含义 |
| --- | --- |
| `sample_count` | 区间内实际存在的设备日记录数 |
| `average_high_risk_probability` | 区间高风险概率平均值 |
| `average_confidence` | 区间置信度平均值 |
| `dominant_discriminator` | 区间平均权重最高的判别器 |
| `average_discriminator_weights` | 四个判别器的区间平均权重 |

> *示例说明：查看设备连续 7 个已发布日期的稳定风险画像。*

```
curl "http://127.0.0.1:8000/api/device/profile?device_id=AFZ.511C&start_date=2026-02-01&end_date=2026-02-07"
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-8e1df64e50694539",
  "data": {
    "device_id": "AFZ.511C",
    "station_id": "SS-AFZ",
    "start_date": "2026-02-01",
    "end_date": "2026-02-07",
    "sample_count": 7,
    "average_high_risk_probability": 0.965777,
    "average_confidence": 0.921879,
    "dominant_discriminator": "历史事件判别器",
    "average_discriminator_weights": {
      "operation": 0.054944,
      "history": 0.766822,
      "maintenance": 0.171888,
      "family_profile": 0.006346
    }
  }
}
```


## 2.9 家族缺陷预警

```
GET /api/defect/family-alerts?assessment_date=YYYY-MM-DD&level=G|M|L&limit=N
```

- 请求体：无。
- 接口约束：`assessment_date` 必填且必须存在于已发布的 `full` 模式数据中；`level` 可选，只能为 `G`、`M` 或 `L`；`limit` 可选，默认 20，范围 1 ~ 100。
- 响应体：只返回家族成员数不少于 3 且 Z-score 达到预警阈值的家族。厂家、型号和批次是南瑞方定位家族的主三元组；`family_id` 是该三元组标准化后的稳定哈希标识。

预警等级规则：`Z-score > 3.0` 为 G，`Z-score > 2.0` 为 M，`Z-score > 1.5` 为 L。Z-score 使用“发生过历史缺陷的家族设备占比”计算；`defect_events_per_device` 单独表示平均每台设备的历史缺陷事件数，避免将多次缺陷事件误写成百分比。

| 响应字段 | 含义 |
| --- | --- |
| `assessment_date` | 本次家族统计对应的设备评估日期 |
| `rule_name` / `rule_version` | 家族预警规则名称和版本 |
| `family_key_definition` | 固定为 `manufacturer + model + batch` |
| `eligible_family_count` | 成员数和数据完整度满足计算条件的家族数 |
| `alert_count` | 过滤前达到预警阈值的家族总数 |
| `items[].family_id` | 三元组标准化后生成的稳定家族标识 |

| 响应字段 | 含义 |
| --- | --- |
| `items[].manufacturer` | 厂家，三元组第一项 |
| `items[].model` | 型号，三元组第二项 |
| `items[].batch` | 批次，三元组第三项 |
| `items[].batch_source` | 批次来源；含“推导”时表示由出厂日期生成 |
| `items[].device_count` | 该家族设备总数 |
| `items[].evaluated_device_count` | 当日成功完成设备模型评估的成员数 |
| `items[].defect_count` | 家族历史缺陷事件总数 |
| `items[].defect_device_count` | 家族内发生过历史缺陷的设备数 |
| `items[].defect_rate` | 缺陷设备占比，范围 0～100 |
| `items[].defect_events_per_device` | 平均每台设备的历史缺陷事件数 |
| `items[].z_score` / `level` | Z-score 和 G/M/L 预警等级 |
| `items[].high_risk_device_count` | 当日设备模型判定为异常或高风险的成员数 |
| `items[].average_high_risk_probability` | 家族成员平均高风险概率 |
| `items[].max_high_risk_probability` | 家族成员最高高风险概率 |
| `items[].top_risk_devices` | 家族内风险最高的最多 3 台设备 |
| `items[].data_quality` | `pass` 或 `warning`；批次由日期推导或模型覆盖不足时为 `warning` |

> *示例说明：查询 2026-02-07 的家族缺陷预警。示例数据中，甲方可使用 `manufacturer`、`model`、`batch` 三元组定位对应家族。*

```
curl "http://127.0.0.1:8000/api/defect/family-alerts?assessment_date=2026-02-07&limit=20"
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-bc9d3ae3cb9941db",
  "data": {
    "assessment_date": "2026-02-07",
    "rule_name": "Family Defect Z-Score",
    "rule_version": "family-zscore-1",
    "family_key_definition": "manufacturer + model + batch",
    "minimum_family_size": 3,
    "eligible_family_count": 4,
    "alert_count": 1,
    "device_model_name": "DAMD-Net",
    "device_model_version": "9e1e625b5935",
    "items": [{
      "family_id": "FAM-6DC07281D5696DD5",
```

```
      "manufacturer": "南瑞继保",
      "model": "RCS-931",
      "batch": "202201",
      "batch_source": "demo_seed",
      "device_count": 3,
      "evaluated_device_count": 3,
      "defect_count": 6,
      "defect_device_count": 3,
      "defect_rate": 100.0,
      "defect_events_per_device": 2.0,
      "z_score": 1.632993,
      "high_risk_device_count": 1,
      "average_high_risk_probability": 0.302597,
      "max_high_risk_probability": 0.731531,
      "level": "L",
      "warning_reason": "家族缺陷设备占比显著高于本次有效家族平均水平，Z-score=1.63",
      "top_risk_devices": [{
        "device_id": "AFZ.511C",
        "station_id": "SS-AFZ",
        "state_level": 2,
        "state_name": "abnormal",
        "high_risk_probability": 0.731531
      }],
      "data_quality": "pass"
    }]
  }
}
```

## 2.10 站点日分级评估与风险分

```
POST /api/station/assessment
```

### 请求体

| 字段 | 类型 | 必填 | 约束与含义 |
|---|---|---|---|
| `request_id` | string | 是 | 1～64 字符 |
| `assessment_date` | date | 是 | 已发布数据中的日期 |
| `station_ids` | string[]/null | 否 | 1～100 个不重复站点 ID；不传或 `null` 表示该日全部站点 |

### 响应体

| 字段 | 含义 |
|---|---|
| `station_id` / `station_name` | 站点标识和名称 |
| `state_level` / `state_name` | HSF-Net 四级分类头输出 |
| `risk_score` | HSF-Net 独立风险头输出，0～100 |
| `confidence` | 分类头最高概率 |
| `high_risk_probability` | 异常和高风险两类概率之和 |

| 字段 | 含义 |
| --- | --- |
| `dominant_view` / `view_weights` | 运行、历史检修、基础设施三个视图的主导视图与权重 |
| `data_quality` | `pass` 或 `warning` |

### 接口约束

- `assessment_date` 必须存在于已发布数据中；`station_ids` 最多 100 个、不得重复，未传时返回该日全部站点。
- 日期或站点不存在时返回 HTTP 404；请求体校验失败时返回 HTTP 422。

> *示例说明：对两个站点执行单日分级评估和风险分计算。*

```
curl -X POST http://127.0.0.1:8000/api/station/assessment \
  -H "Content-Type: application/json" \
  -d '{
    "request_id": "demo-station-001",
    "assessment_date": "2026-02-07",
    "station_ids": ["SS-AFZ", "SS-BAZ"]
  }'
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "demo-station-001",
  "data": {
    "assessment_date": "2026-02-07",
    "model_name": "HSF-Net",
    "model_version": "56be09ba8581",
    "items": [{
      "station_id": "SS-AFZ",
      "station_name": "A阜站",
      "state_level": 0,
      "state_name": "normal",
      "risk_score": 56.456955,
      "confidence": 0.446408,
      "high_risk_probability": 0.251659,
      "dominant_view": "运行状态视图",
      "view_weights": {
        "operation": 0.430679,
        "history_maintenance": 0.296088,
        "infrastructure": 0.273234
      },
      "data_quality": "pass"
    }]
  }
}
```

`risk_score` 和 `state_level` 来自不同输出头，一个是连续分，一个是四级分类，不要互相替代。权重只表示模型当次更依赖哪一类特征，不等于可审计的故障原因定位。

## 2.11 单站点单日查询

```
GET /api/station/query?assessment_date=YYYY-MM-DD&station_id=...
```

- 请求体：无。
- 接口约束：查询参数 `assessment_date` 和 `station_id` 均必填；站点或日期无数据时返回 HTTP 404。
- 响应体：与 2.10 节相同，`items` 只有该站点。

> *示例说明：查询单个站点的单日结果。*

```
curl "http://127.0.0.1:8000/api/station/query?assessment_date=2026-02-07&station_id=SS-AFZ"
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-e18a868792864caa",
  "data": {
    "assessment_date": "2026-02-07",
    "model_name": "HSF-Net",
    "model_version": "56be09ba8581",
    "items": [{
      "station_id": "SS-AFZ",
      "station_name": "A阜站",
      "state_level": 0,
      "state_name": "normal",
      "risk_score": 56.456955,
      "confidence": 0.446408,
      "high_risk_probability": 0.251659,
      "dominant_view": "运行状态视图",
      "view_weights": {
        "operation": 0.430679,
        "history_maintenance": 0.296088,
        "infrastructure": 0.273234
      },
      "data_quality": "pass"
    }]
  }
}
```


## 2.12 站点时间区间画像

```
GET /api/station/profile?station_id=...&start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
```

- 请求体：无。
- 接口约束：`station_id`、`start_date`、`end_date` 均必填，开始日期不得晚于结束日期，且区间内至少存在一条该站点的已发布日数据。
- 响应体：对区间内已发布的站点日结果进行聚合，不是另一个模型；`data` 字段如下。

| 响应字段 | 含义 |
| --- | --- |
| `sample_count` | 区间内实际站点日记录数 |
| `average_risk_score` | 站点风险分平均值 |
| `average_high_risk_probability` | 高风险概率平均值 |
| `average_confidence` | 分类置信度平均值 |

| 响应字段 | 含义 |
| --- | --- |
| `dominant_view` / `average_view_weights` | 区间主导视图和三视图平均权重 |

> *示例说明：查看站点连续 7 个已发布日期的风险画像。*

```
curl "http://127.0.0.1:8000/api/station/profile?station_id=SS-AFZ&start_date=2026-02-
01&end_date=2026-02-07"
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-99fbf27d85f24527",
  "data": {
    "station_id": "SS-AFZ",
    "station_name": "A 阜站",
    "start_date": "2026-02-01",
    "end_date": "2026-02-07",
    "sample_count": 7,
    "average_risk_score": 58.987236,
    "average_high_risk_probability": 0.24667,
    "average_confidence": 0.428132,
    "dominant_view": "运行状态视图",
    "average_view_weights": {
      "operation": 0.411746,
      "history_maintenance": 0.285281,
      "infrastructure": 0.302973
    }
  }
}
```


## 2.13 单日风险总览

```
GET /api/risk/overview?assessment_date=YYYY-MM-DD&top_n=N
```

- 请求体：无。
- 接口约束：`assessment_date` 必填并已发布；`top_n` 可选，默认 5，范围 1 ~ 50。
- 响应体：只汇总同一日期的设备和站点评估结果，不是独立预测模型；`data` 字段如下。

| 响应字段 | 含义 |
| --- | --- |
| `device_state_counts` / `station_state_counts` | 设备/站点四级数量 |
| `average_device_high_risk_probability` | 当日设备高风险概率平均值 |
| `average_station_risk_score` | 当日站点风险分平均值 |
| `top_risk_devices` / `top_risk_stations` | 按设备高风险概率/站点风险分排序的前 N 项 |

> *示例说明：返回当日分级分布和风险最高的 3 个对象。*

```
curl "http://127.0.0.1:8000/api/risk/overview?assessment_date=2026-02-07&top_n=3"
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "req-5d71578d49fd48ae",
  "data": {
    "assessment_date": "2026-02-07",
    "device_state_counts": {"normal": 3, "watch": 8, "abnormal": 1, "high_risk": 0},
    "station_state_counts": {"normal": 5, "watch": 0, "abnormal": 0, "high_risk": 0},
    "average_device_high_risk_probability": 0.208636,
    "average_station_risk_score": 49.66228,
    "top_risk_devices": [{
      "device_id": "AFZ.511C",
      "station_id": "SS-AFZ",
      "state_name": "abnormal",
      "high_risk_probability": 0.966921
    }],
    "top_risk_stations": [{
      "station_id": "SS-BAZ",
      "station_name": "B站",
      "state_name": "normal",
      "risk_score": 61.423092
    }]
  }
}
```

## 2.14 经济调度建议

```
POST /api/economic-dispatch
```

### 请求体

| 字段 | 类型 | 必填 | 约束与含义 |
| --- | --- | --- | --- |
| `request_id` | string | 是 | 1～64 字符 |
| `assessment_date` | date | 是 | 已发布站点数据日期 |
| `station_ids` | string[]/null | 否 | 1～100 个不重复站点 ID；`null` 表示该日全部站点 |
| `supply_budget_mw` | number | 是 | 可用调节总预算，大于等于 0 |
| `economic_parameters` | object | 否 | 经济参数；不传时使用内置基线 |

`economic_parameters` 字段：

| 字段 | 默认值 | 约束 | 含义 |
| --- | --- | --- | --- |
| `load_growth_rate` | 0.08 | 0～1 | 基础负荷增长率 |
| `capacity_multiplier` | 1.25 | 0～5，不含 0 | 历史负荷容量系数 |
| `dispatch_cost_base` | 0.30 | 大于等于 0 | 基础调节成本 |
| `risk_cost_base` | 0.20 | 大于等于 0 | 基础风险成本 |
| `risk_cost_weight` | 1.10 | 大于等于 0 | 站点组合风险惩罚权重 |
| `shortfall_cost_base` | 1.25 | 大于等于 0 | 基础缺额损失 |
| `shortfall_criticality_weight` | 1.75 | 大于等于 0 | 站点重要度的缺额损失权重 |

### 响应体

| 字段 | 含义 |
| --- | --- |
| `model_name` | 当前为 `RA-MOD` |
| `solver_status` | `optimal` 表示线性规划求解成功；求解失败时可回退贪心方案 |

| 字段 | 含义 |
| --- | --- |
| `station_count` | 参与调度的站点数 |
| `required_adjustment_mw` | 站点需求调节量合计 |
| `allocated_adjustment_mw` | 建议分配调节量合计 |
| `unserved_adjustment_mw` | 未满足调节量合计 |
| `parameter_source` | `built_in_baseline` 或 `request_override` |
| `economic_parameters` | 本次实际生效的经济参数 |
| `items[].max_adjustable_mw` | 考虑容量、状态和检修后的可调上限 |
| `items[].combined_risk_index` | 0～1 的组合风险索引 |
| `items[].recommended_allocation_mw` | 给该站点的建议分配量 |
| `items[].unserved_adjustment_mw` | 该站点未满足量 |
| `items[].adjustment_service_rate` | 建议分配量/需求调节量 |
| `items[].dispatch_priority` | 风险调整后的站点优先级 |
| `items[].dispatch_reason` | 本次是否分配的原因说明 |

### 接口约束

- `assessment_date` 必须存在已发布站点数据中；`station_ids` 最多 100 个、不得重复，未传时使用该日全部站点。
- `supply_budget_mw` 必须大于等于 0，经济参数必须满足上表范围；校验失败时返回 HTTP 422。

> *示例说明：在总预算 10 MW 下，对两个站点使用显式经济参数生成调度建议。*

```
curl -X POST http://127.0.0.1:8000/api/economic-dispatch \
  -H "Content-Type: application/json" \
  -d '{
    "request_id": "demo-dispatch-001",
    "assessment_date": "2026-02-07",
    "station_ids": ["SS-AFZ", "SS-BAZ"],
    "supply_budget_mw": 10.0,
    "economic_parameters": {
      "load_growth_rate": 0.08,
      "capacity_multiplier": 1.25,
      "dispatch_cost_base": 0.30,
      "risk_cost_base": 0.20,
      "risk_cost_weight": 1.10,
      "shortfall_cost_base": 1.25,
      "shortfall_criticality_weight": 1.75
    }
  }'
```

```
{
  "code": "0",
  "message": "success",
  "request_id": "demo-dispatch-001",
  "data": {
    "assessment_date": "2026-02-07",
    "model_name": "RA-MOD",
    "solver_status": "optimal",
    "station_count": 2,
    "supply_budget_mw": 10.0,
    "required_adjustment_mw": 10.41168,
    "allocated_adjustment_mw": 5.64288,
    "unserved_adjustment_mw": 4.7688,
    "parameter_source": "request_override",
    "economic_parameters": {
      "load_growth_rate": 0.08,
      "capacity_multiplier": 1.25,
      "dispatch_cost_base": 0.3,
      "risk_cost_base": 0.2,
      "risk_cost_weight": 1.1,
      "shortfall_cost_base": 1.25,
      "shortfall_criticality_weight": 1.75
    },
    "items": [{
      "station_id": "SS-BAZ",
      "station_name": "B爱站",
      "max_adjustable_mw": 5.64288,
      "combined_risk_index": 0.39,
      "recommended_allocation_mw": 5.64288,
      "unserved_adjustment_mw": 0.0,
      "adjustment_service_rate": 1.0,
      "dispatch_priority": 1,
      "dispatch_reason": "allocated by avoided shortfall loss under cost and risk
constraints"
    }]
  }
}
```


当前 RA-MOD 使用站点日数据、站点风险、容量代理值和经济参数做线性优化。它不执行潮流计算，也没有按拓扑图自动将电网划分为调度区域。`station_id` 来自 XML 中的设备-容器-站点关系，接口对请求选中的站点集合优化。结果是调度建议，不直接下发控制指令。

## 2.15 模型和数据版本查询

```
GET /api/models
```

- 请求体：无。
- 接口约束：无；可选 `X-Request-ID` 请求头。
- 响应体：HTTP 200；返回实际加载的模型、推理设备和当前数据版本，`data` 字段如下。

| 响应字段 | 含义 |
| --- | --- |
| `inference_device` | 当前 `cpu`、`mps` 或 `cuda:N` |
| `training_enabled` | 固定为 `false` |
| `active_feature_version` | 当前活跃数据版本、日期和行数 |

| 响应字段 | 含义 |
| --- | --- |
| `models[].name` / `version` | 模型名称和文件校验版本 |
| `models[].purpose` | 模型用途 |
| `models[].components` | 设备判别器或站点视图定义 |
| `models[].family_key_definition` | 家族预警规则使用的三元组定义 |

> *示例说明：联调时确认服务实际加载的模型、推理设备和数据版本。*

```
curl http://127.0.0.1:8000/api/models
```


```
{
  "code": "0",
  "message": "success",
  "request_id": "req-0030235c5e20453d",
  "data": {
    "inference_device": "cpu",
    "training_enabled": false,
    "active_feature_version": {
      "version": "20260207-demo-seed-081008-f053a8",
      "business_date": "2026-02-07",
      "run_mode": "full",
      "device_rows": 84,
      "station_rows": 35,
      "published_at": "2026-08-11T08:10:08+00:00"
    },
    "models": [
      {
        "name": "DAMD-Net",
        "version": "9e1e625b5935",
        "purpose": "device_state_assessment",
        "components": {
          "operation": "运行波动判别器",
          "history": "历史事件判别器",
          "maintenance": "检修恢复判别器",
          "family_profile": "设备画像判别器"
        }
      },
      {
        "name": "HSF-Net",
        "version": "56be09ba8581",
        "purpose": "station_state_and_risk_assessment",
        "components": {
          "operation": "运行状态视图",
          "history_maintenance": "历史检修视图",
          "infrastructure": "基础设施视图"
        }
      },
      {
        "name": "RA-MOD",
        "version": "rule-lp-1",
        "purpose": "risk_aware_economic_dispatch",
        "parameter_source": "request_and_config"
      },
      {
        "name": "Family Defect Z-Score",
        "version": "family-zscore-1",
        "purpose": "family_defect_alert",
        "family_key_definition": "manufacturer + model + batch"
      }
    ]
```

```
  }
}
```

## 2.16 错误响应

| HTTP | code | 含义 |
| --- | --- | --- |
| 400 | `INVALID_REQUEST` / `INVALID_DATA` | 参数组合或处理数据不合法 |
| 404 | `DATA_NOT_FOUND` | 任务、日期、设备或站点不存在 |
| 422 | `INVALID_REQUEST` | Pydantic 请求体/查询参数校验失败 |
| 503 | `FEATURE_DATA_NOT_READY` | 尚无可用特征版本，或家族接口读取的不是包含三元组字段的 `full` 版本 |

> *示例说明：请求一个当前数据版本中不存在的设备。*

```
{
  "code": "DATA_NOT_FOUND",
  "message": "Unknown device IDs for the requested date range: ['UNKNOWN-DEVICE']",
  "request_id": "req-9bd38bb99f384d23",
  "data": null
}
```

# 3. 部署须知及待对接部分

## 3.1 交付的容器结构

当前已提供：

```
.dockerignore
api_service/Dockerfile
api_service/compose.yaml
api_service/compose.gpu.yaml
api_service/.env.example
api_service/requirements.txt
api_service/models/*.pt
api_service/scripts/docker_demo.sh
```

API 和 Worker 使用同一个镜像、不同启动命令，共享 `api_service/runtime/` 持久化目录。模型文件已放入镜像可复现路径，并保存 SHA-256 校验值。

已实际构建 `power-grid-model-api:1.0.0-cpu` 与 `power-grid-model-api:1.0.0-cu128` 两个镜像。CPU 版已完成 Compose 启动和全部 API/Worker 联调；GPU 版已完成 `linux/amd64` 跨架构构建、PyTorch CUDA 12.8 导入和模型产物加载。RTX 3090 实机推理需在南瑞方目标 Linux 服务器最终验收。

## 3.2 具体软硬件要求

### 生产建议基线

| 类别 | 最低配置 | 建议采购/部署配置 |
| --- | --- | --- |
| 架构 | Linux `x86_64` | Ubuntu 22.04 LTS 或 24.04 LTS `x86_64` |
| API CPU | 4 vCPU | 8 vCPU |
| API 内存 | 8 GB | 16 GB |
| Worker CPU | 8 vCPU | 16 vCPU |
| Worker 内存 | 32 GB | 64 GB |
| 持久化存储 | 200 GB SSD | 500 GB NVMe，再根据 XML/DT 日批次保留周期扩容 |
| GPU | 日级推理可不配 GPU | NVIDIA RTX 3090 24 GB 或同级更高显存 GPU，单卡即可 |
| 网络 | API 端口与 DB 网络可达 | 经 API 网关对南瑞方开放 HTTPS，Worker 不对外开放端口 |

上述 CPU、内存和存储是当前 4.19 GB 原始处理包及日级模型的工程基线，最终采购前应在目标数据量下进行一次全链路压测。当前两个表格神经网络规模较小，GPU 不是日级在线推理的强制条件；如服务器已确定为 3090，可使用 GPU 配置。

### NVIDIA/CUDA 要求

| 项目 | 当前容器基线 |
| --- | --- |
| Python | 3.11 |
| PyTorch | 2.11.0 |
| PyTorch CUDA wheel | `cu128` |
| CUDA 用户态运行库 | 随容器/PyTorch wheel 安装，宿主机不需单独安装 CUDA Toolkit |
| Linux NVIDIA 驱动最低值 | `570.26` |
| 建议驱动 | R570 且不低于 `570.124.06` |
| NVIDIA Container Toolkit | `1.19.1` 或部署时经验收的后续兼容版 |
| GPU 选择 | `NVIDIA_GPU_ID` 选择宿主机物理 GPU；单卡容器内使用 `TORCH_DEVICE=cuda`、`GPU_ID=0` |

NVIDIA 官方 CUDA 12.8 发布说明列出 Linux 驱动的兼容下限与对应工具包驱动版本；部署时按 CUDA 12.8 Release Notes 和 NVIDIA Container Toolkit Installation Guide 复核。

### Python 依赖

```
FastAPI 0.141.1
Uvicorn 0.52.1
Pydantic 2.13.4
SQLAlchemy 2.0.51
NumPy 2.4.6
Pandas 3.0.3
SciPy 1.17.1
scikit-learn 1.9.0
Joblib 1.5.3
PyTorch 2.11.0+cu128
```

PyTorch 2.11.0 `cu128` 安装组合可在 PyTorch Previous Versions 核对。由于 `.pt` 模型产物同时包含 scikit-learn 预处理器，生产镜像必须使用锁定依赖，不建议在目标服务器上临时升级。

## 3.3 Docker 启动

在项目根目录复制环境文件：

```
cp api_service/.env.example api_service/.env
```

CPU 环境首次构建并准备演示数据：

```
docker compose -f api_service/compose.yaml build model-api
docker compose -f api_service/compose.yaml run --rm --no-deps \
  model-api python -m api_service.demo.prepare_demo_data
docker compose -f api_service/compose.yaml up -d
```

3090 GPU 启动：

```
docker compose \
  -f api_service/compose.yaml \
  -f api_service/compose.gpu.yaml \
  up --build -d
```

验证：

```
docker compose -f api_service/compose.yaml ps
curl http://127.0.0.1:8000/health/live
curl http://127.0.0.1:8000/health/ready
docker compose -f api_service/compose.yaml exec -T \
  model-api python api_service/demo/run_smoke_test.py
```

Swagger 地址为 `http://127.0.0.1:8000/docs`。日志与停止命令：

```
docker compose -f api_service/compose.yaml logs -f --tail=100
docker compose -f api_service/compose.yaml down
```

3090 环境启动后，应在目标服务器执行：

```
docker compose \
  -f api_service/compose.yaml \
  -f api_service/compose.gpu.yaml \
  exec -T model-api python api_service/scripts/verify_environment.py
```

关键验收值为 `torch=2.11.0+cu128`、`cuda_runtime=12.8`、`cuda_available=True` 且 GPU 名称为 RTX 3090。完整 Docker 使用与离线交付说明见 `docs/docker_deployment_guide.md`。

## 3.4 宿主机 Python 调试

```
conda activate <已配置的 Python 环境>
python -m pip install -r api_service/requirements.txt
python -m api_service.demo.prepare_demo_data
```

分别在两个终端启动：

```
python -m uvicorn api_service.app.main:app --host 127.0.0.1 --port 8000

python -m api_service.worker
```

然后执行所有联调报文：

```
python api_service/demo/run_smoke_test.py
```

## 3.5 部署时必须确认的信息

| 类别 | 待华工方与南瑞方确认的内容 |
| --- | --- |
| 服务器 | Linux 发行版、`x86_64`/`aarch64`、CPU、内存、磁盘 |

| 类别 | 待华工方与南瑞方确认的内容 |
| --- | --- |
| GPU | 型号、数量、显存、允许使用的 GPU 编号和 `nvidia-smi` 输出 |
| 容器 | Docker/Containerd 版本、Compose 版本、NVIDIA Container Toolkit |
| DB | 类型、版本、schema、表/视图名、驱动、字符集、时区、只读权限 |
| 文件区 | XML/DT 上传协议、共享路径、用户/UID 权限、保留周期 |
| 网络安全 | API 网关地址、HTTPS 证书、认证方式、防火墙和 DB 白名单 |
| 业务规模 | 每日 XML/DT 大小、设备/站点数、保留周期、API 并发与 SLA |

DB 未确定前，可使用 `operation_only` 完成 XML/DT 全链路联调；生产 `full` 模式必须在 DB 类型、驱动和 6 类逻辑表映射确认后进行验收。